UI仕様書: Petri Netエディタスクリーン

1. 概要

1.1 目的

MAIMLファイルをフルスクリーンで編集するためのエディタ画面。ProjectManagementまたはFormatManagementから呼び出され、Petri Net形式でMAIMLファイルを視覚的に編集する。

1.2 使用場面

- プロジェクト管理画面からMAIMLファイルを編集する場合（editMode='project'）
- フォーマット管理画面からフォーマットファイルを編集する場合（editMode='format'）

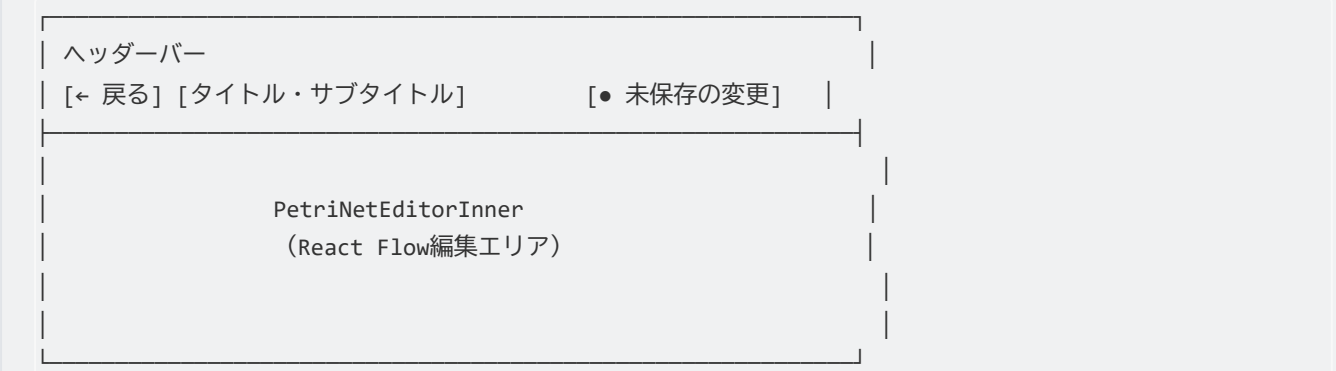
1.3 アクセス権限

- **実験管理者（Experiment Admin）**: 読み取り・編集可能

---

2. 画面構成

2.1 レイアウト構造



2.2 UI要素

ヘッダーバー

- **戻るボタン**: 編集画面を閉じて元の画面に戻る
- **タイトル**:
  - editMode='project': 「実験フロー編集 - {displayTitle}」
  - editMode='format': 「フォーマット編集 - {displayTitle}」
- **サブタイトル**: displaySubtitle（存在する場合のみ表示）
- **未保存変更インジケーター**: hasUnsavedChangesがtrueの場合に表示

エディタ本体

- **PetriNetEditorInner**: Petri Net編集コンポーネント（ReactFlowProviderでラップ）

ローディング画面

- **Spin**: ファイル読み込み中に表示

---

3. 機能仕様

3.1 Props定義

```
interface PetriNetEditorScreenProps {
  /** 編集パラメータ（共通型） */
  params: MaimlEditorParams;
  /** 画面を閉じるコールバック */
  onClose: () => void;
}

// MaimlEditorParams型の内容
interface MaimlEditorParams {
  editMode: 'project' | 'format';
  editingFileName: string;
```

```
  originalFileName?: string;
  sourceS3Key: string;
  displayTitle: string;
  displaySubtitle?: string;
  documentId?: string;
  description?: string;
}
```

3.2 State管理

```
// ローディング状態
const [isLoading, setIsLoading] = useState<boolean>(true);

// MAIMLファイルの内容（文字列）
const [maimlContent, setMaimlContent] = useState<string | null>(null);

// 未保存の変更があるか
const [hasUnsavedChanges, setHasUnsavedChanges] = useState<boolean>(false);
```

3.3 主要機能

3.3.1 MAIMLファイルの読み込み

トリガー: useEffect（sourceS3Keyが変更されたとき）

処理フロー:

1. `setIsLoading(true)`
2. `getFileContentAsText(sourceS3Key)` でS3からファイル取得
3. 取得成功:
   - `setMaimlContent(content)` - 内容を設定
   - コンソールログ: ✅ 読み込み成功、文字列長を出力
4. 取得失敗またはnull:
   - `setMaimlContent(null)` - 新規作成モード
   - コンソールログ: ✅ 新規作成モード

5. `setIsLoading(false)`

新規作成モード:

- ファイルが存在しない場合（404エラーやnull）は、maimlContent=nullで新規作成として扱う

3.3.2 戻るボタンの処理

トリガー: 戻るボタンクリック

処理フロー:

1. `hasUnsavedChanges` をチェック
2. 未保存変更がある場合:
   - `window.confirm()` で確認ダイアログ表示
   - メッセージ:「保存されていない変更があります。破棄して戻りますか？」
   - ユーザーが「OK」: `onClose()` を呼び出し
   - ユーザーが「キャンセル」: 何もしない
3. 未保存変更がない場合:
   - `onClose()` を呼び出し

3.3.3 コンテンツ変更の検知

トリガー: PetriNetEditorInnerからの通知（未実装、将来的な拡張ポイント）

処理フロー:

1. `hasUnsavedChanges` がfalseの場合のみ
2. `setHasUnsavedChanges(true)` に変更
3. コンソールログ: 📝 変更を検知

注: 現在は `handleContentChange` コールバックが定義されているが、PetriNetEditorInnerに渡されていない

---

4. データ仕様

4.1 Props型定義

```
interface PetriNetEditorScreenProps {
  params: MaimlEditorParams;
  onClose: () => void;
}

interface MaimlEditorParams {
  editMode: 'project' | 'format';
  editingFileName: string;
  originalFileName?: string;
  sourceS3Key: string;
  displayTitle: string;
  displaySubtitle?: string;
  documentId?: string;
  description?: string;
}
```

4.2 State型定義

```
const isLoading: boolean;
const maimlContent: string | null;
const hasUnsavedChanges: boolean;
```

5. API連携

5.1 ファイル取得API

関数	説明	パラメータ	戻り値
`getFileContentAsText`	S3からMAIMLファイルをテキストとして取得	`s3Key: string`	`Promise<string | null>`

エラーハンドリング:

- ファイルが存在しない場合はnullを返す
- エラーが発生した場合もnullを返し、新規作成モードとして続行

---

6. 子コンポーネント

6.1 PetriNetEditorInner

Props:

```
{
  maimlData: string | null;
  editingFileName: string;
  editMode: 'project' | 'format';
  originalFileName?: string;
  sourceS3Key: string;
  initialDocumentId?: string;
  initialDescription?: string;
}
```

役割: Petri Net編集の本体（React Flowを使用したノード・エッジ編集）

6.2 ReactFlowProvider

役割: React Flowのコンテキストを提供（PetriNetEditorInnerをラップ）

---

7. スタイリング

7.1 画面全体

- **width**: 100vw
- **height**: 100vh
- **display**: flex（縦方向）

- **overflow**: hidden
- **backgroundColor**: #ffffff

7.2 ヘッダーバー

- **padding**: 12px 16px
- **borderBottom**: 1px solid #f0f0f0
- **display**: flex（横方向）
- **alignItems**: center
- **gap**: 16px
- **backgroundColor**: white
- **boxShadow**: 0 2px 8px rgba(0,0,0,0.06)
- **zIndex**: 10

7.3 タイトル

- **fontSize**: 18px
- **fontWeight**: 600
- **color**: #262626

7.4 サブタイトル

- **fontSize**: 14px
- **color**: #8c8c8c

7.5 未保存変更インジケーター

- **padding**: 4px 12px
- **backgroundColor**: #fff7e6
- **border**: 1px solid #ffd591
- **borderRadius**: 4px
- **fontSize**: 14px
- **color**: #d46b08

7.6 ローディング画面

- **display**: flex
- **justifyContent**: center
- **alignItems**: center
- **height**: 100vh
- **backgroundColor**: #f0f2f5

---

8. エラーハンドリング

8.1 ファイル読み込みエラー

発生条件:

- S3からのファイル取得に失敗
- ファイルが存在しない

対応:

- エラーをコンソールに出力（❌マーク付き）
- `maimlContent = null` として新規作成モードで続行
- ユーザーにエラーメッセージは表示しない（新規作成として扱う）

8.2 未保存変更の破棄確認

発生条件:

- 未保存変更がある状態で戻るボタンをクリック

対応:

- ブラウザネイティブの `window.confirm()` で確認
- メッセージ:「保存されていない変更があります。破棄して戻りますか？」

---

9. 実装ファイル

9.1 メインファイル

- **パス**: `frontend/src/components/editorv2/PetriNetEditorScreen.tsx`
- **行数**: 約210行
- **言語**: TypeScript + React

9.2 関連ファイル

- `frontend/src/components/editorv2/PetriNetEditorInner.tsx` - エディタ本体
- `frontend/src/services/fileAccessService.ts` - ファイルアクセスサービス
- `frontend/src/types/maimlEditor.ts` - 型定義

9.3 依存ライブラリ

- `react` - Reactフレームワーク
- `antd` - Ant Designコンポーネント（Button, Spin, message）
- `reactflow` - React Flow（ReactFlowProvider）

---

10. 特記事項

10.1 旧Propsの廃止

以前は `projectId`, `projectName`, `projectOverview` を個別のPropsとして受け取っていたが、現在は `MaimlEditorParams` 型に一本化されている。

10.2 新規作成モード

ファイルが存在しない場合（S3から取得できない場合）は、`maimlContent = null`として新規作成モードで動作する。PetriNetEditorInnerが空のPetri Netを生成する。

10.3 未保存変更の検知

現在は`handleContentChange`コールバックが定義されているが、PetriNetEditorInnerに渡されていない。将来的には、PetriNetEditorInner側でコンテンツが変更されたときにこのコールバックを呼び出すことで、未保存変更の検知が可能になる。

10.4 ReactFlowProvider

React Flowを使用するため、PetriNetEditorInnerを`<ReactFlowProvider>`でラップする必要がある。これにより、React Flowのコンテキスト（ノード・エッジの状態管理）が提供される。

10.5 フルスクリーン表示

この画面は100vw x 100vhのフルスクリーン表示となり、元の画面（ProjectManagement / FormatManagement）を完全に覆う。

---

11. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースでUI仕様書を新規作成	Cline

---

12. 参考情報

12.1 呼び出し元

- **ProjectManagement**: プロジェクト内のMAIMLファイルを編集
- **FormatManagement**: フォーマットファイルを編集

12.2 関連UI仕様書

- `ui_petri_net_editor.md` - PetriNetEditorInnerの詳細仕様（Phase 6-2で作成予定）
- `ui_project_management.md` - プロジェクト管理画面
- `ui_format_management.md` - フォーマット管理画面